

Interview Ideas and Shape Better Work

Session 21 | Product and Delivery Teams | 3 hours

GitHub Copilot is required

Before the session:

- Copy the supplied project skill to `.github/skills/decision-interview/SKILL.md`.
- Start a new project session.
- Confirm `decision-interview` under **Customize** → **Skills**.
- Run `/decision-interview` with the short test request.

Stop and resolve access if any check fails.

Start with the actual problem

"Make request status clearer so people stop asking support."

This is an idea. It is not sprint-ready work.

It hides choices about users, scope, data, behavior, ownership, and approval.

The shaping flow

Vague request

- > project skill installation
- > GitHub Copilot decision interview
- > human decisions
- > Markdown decision brief
- > GitHub issue proposal
- > acceptance criteria
- > definition of done

Do not jump from one sentence to an issue template.

Interview as a decision tree

Each answer can raise new questions.

Ask the current **frontier**: every decision the owner can make now without guessing about an unsettled decision.

Questions that depend on this round wait for the next round.

Work in rounds

Round 1: outcome, affected user, current pain
Round 2: scope, current state, constraints
Round 3: success, failure, edge cases
Round 4: owner, reviewer, approval, rollout

Ask GitHub Copilot to recompute the next frontier from the confirmed answers.

Give a recommendation

For each question:

1. State the decision clearly.
2. Offer concrete choices when useful.
3. Recommend an answer and explain the tradeoff.
4. Wait for the request owner to decide.

GitHub Copilot frames the choice. **A person owns the choice.**

Facts and decisions are different

Use approved evidence for facts	Ask people for decisions
Existing workflow	Which users are in scope
Current labels or fields	Which behavior should change
Known technical limits	Which tradeoff to accept
Policy and data rules	Who approves release

Mark a fact as unverified. Do not present it as settled.

Six branches must close

- **Scope:** who and what are included?
- **Constraints:** what limits the solution?
- **Success:** what observable result matters?
- **Failure:** what can go wrong, and what should happen?
- **Ownership:** who decides and who maintains it?
- **Approval:** who reviews the brief and accepts the work?

An open branch stays visible.

The decision brief

A useful brief records:

- problem and target outcome;
- users, scope, and exclusions;
- constraints and dependencies;
- success and failure measures;
- settled decisions and open questions;
- owner, reviewer, and approval gate.

Keep it short enough to review in one sitting.

Create a GitHub issue proposal

Ask GitHub Copilot to draft:

- user and outcome;
- bounded scope and non-goals;
- acceptance criteria;
- dependencies and risks;
- owner and reviewer;
- definition of done.

Compare the draft with the approved brief.

Session 22 creates the approved issues through GitHub MCP.

Reopen only the affected branch

Late evidence changes requester and support visibility.

Approved brief

- + new visibility constraint
- reopen visibility decisions
- update affected criteria
- preserve unrelated decisions

Do not restart the interview or rewrite settled scope without evidence.

Verify in a fresh session

Give a new session only:

- the revised decision brief;
- the revised issue proposal.

Ask it to find contradictions, unresolved owners, and acceptance criteria with no source decision.

Acceptance criteria describe evidence

Weak:

Status is easy to understand.

Testable:

A requester can see the current status and last updated time for each submitted request.

Include a failure case when the feature can mislead, expose data, or block progress.

Definition of done closes delivery

Acceptance criteria describe product behavior.

Definition of done covers the work needed to release and hand over safely:

- criteria reviewed;
- permissions checked;
- supported states documented;
- validation evidence attached;
- product and service owners approve.

Azure Boards update paths

The interview always runs with **GitHub Copilot**.

Azure Boards path	Method
MCP	Copilot prepares and reviews the approved update before the MCP tool sends it
Browser	Copilot prepares the exact values, then the learner enters and verifies them

There is no static or no-Copilot route.

Lab deliverables

1. The installed `decision-interview` project skill.
2. A **Copilot interview record** with the reopened branch.
3. A revised **decision brief**.
4. An aligned **GitHub issue proposal**.
5. A fresh-session readiness result.

The request owner approves the brief. A delivery reviewer checks readiness.

Stop conditions

Stop when:

- GitHub Copilot access does not pass preflight;
- the input is not approved or sanitized;
- a required decision owner is absent;
- policy or data boundaries are unclear;
- the team cannot state how success or failure will be observed.

Record content blockers. Resolve Copilot access before starting the exercise.